Computer Networks 271 (2025) 111597

Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet

Robust offloading strategy in VEC with computation uncertainty and imperfect CSI☆

Pengcheng Qian, Liang Wang *, Jiamin Fan, Zhenzheng Shi, Mengge Li

School of Computer Science, Shaanxi Normal University, Xi'an 710119, China

ARTICLE INFO

Keywords:
Vehicular edge computing
Computation uncertainty
Imperfect CSI
Robust design
One-to-many matching

ABSTRACT

In vehicular edge computing (VEC), connected and automated vehicles (CAVs) generate massive data and require efficient processing capabilities to meet the real-time demands of safety-critical applications. Traditional task offloading approaches in VEC face challenges due to inherent computation uncertainties, exacerbated by high-speed vehicle mobility and its impact on channel state information (CSI). This paper addresses these challenges by proposing a robust task offloading scheme that jointly optimizes task partitioning and offloading decisions. Our method focuses on minimizing the total task processing delay by simultaneously considering both communication and computation uncertainties. We model this problem as a mixed-integer nonlinear programming (MINLP) problem and introduce an iterative framework that transforms the complex one-to-many offloading issue into a matching problem. Utilizing a task offloading strategy based Gale-Shapley algorithm (TOGS), our approach ensures efficient task distribution across multiple edge servers. Simulation results demonstrate that our proposed scheme significantly reduces the total task processing delay, outperforming the KM-Hybrid and KM-Edge schemes by approximately 41% and 54%, respectively. The robustness of our approach is validated under various computational complexity scenarios, making it suitable for real-world VEC applications.

1. Introduction

Connected and automated vehicles (CAVs) are envisioned as one of the key technologies in 6G for improving traffic safety and transportation efficiency [1–3]. By integrating vehicle networking and autonomous driving technologies, CAVs can perceive their surrounding environment, collect data on roads, vehicles, and pedestrians, and make intelligent decisions based on this information [4]. In this process, CAVs will generate terabytes of data and large amounts of processing tasks everyday [5]. Thus, efficient processing capabilities are crucial for the successful operation of CAV systems. However, the computational demands of processing this high volume data, which includes tasks such as object recognition, path planning, and decision-making, can quickly overwhelm the onboard computing resources of individual vehicles [6]. This issue creates a pressing need for innovative solutions that can handle the data deluge while meeting the strict latency requirements of safety-critical applications in CAVs.

Fortunately, mobile edge computing (MEC) offers a promising solution to address this challenge [7]. MEC aims to significantly reduce data transmission and processing latency by deploying computing and storage resources to the network edge [8]. This technology brings cloud computing capabilities closer to end users and data sources, not only improving system response time but also enhancing data processing capabilities and network efficiency, and providing crucial support for applications like CAVs that need high real-time data processing requirements [9]. The integration of Internet of Vehicles and MEC gives rise to vehicular edge computing (VEC) [10]. Within this framework, CAVs can offload their tasks to nearby edge servers to alleviate their own computational burden. Therefore, developing efficient task offloading strategies becomes essential for optimizing the overall system performance and processing efficiency.

However, despite its promise, task offloading in VEC is far from trivial. In real-world scenarios, task offloading decisions must be made in highly dynamic environments where both wireless communication conditions and edge server workloads fluctuate rapidly. These dynamics introduce significant uncertainties into the offloading process, especially in terms of channel state information (CSI) and the availability of computational resources at the edge. If not properly addressed, such uncertainties can lead to degraded system performance, increased

☆ This work is supported by the National Natural Science Foundation of China under Grant 62071283.
* Corresponding author.
E-mail addresses: qianpengcheng@snnu.edu.cn (P. Qian), wangliang@snnu.edu.cn (L. Wang), fanjiamin@snnu.edu.cn (J. Fan), shizhenzheng@snnu.edu.cn (Z. Shi), 192339@snnu.edu.cn (M. Li).

https://doi.org/10.1016/j.comnet.2025.111597
Received 8 January 2025; Received in revised form 29 May 2025; Accepted 30 July 2025
Available online 9 August 2025
1389-1286/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

task latency, or even task failure. This makes it imperative to design task offloading strategies that are not only efficient but also robust against such uncertainties. The existing approaches face several critical limitations that prevent them from effectively addressing the challenges in practical VEC scenarios:

- Single-uncertainty consideration: Most existing works only consider either computation uncertainty or channel uncertainty in isolation. This oversimplified assumption fails to capture the reality of VEC systems where both uncertainties coexist and couple with each other. This separate treatment of these uncertainties leads to sub-optimal solutions that may perform poorly when both types of uncertainties are present simultaneously.
- Inefficient resource utilization: A significant portion of existing works fail to fully exploit the available computational resources in VEC. Binary offloading schemes, while simple to design, can only offload computational tasks to a single server, which significantly limits resource utilization efficiency. While partial offloading schemes demonstrate superior resource utilization efficiency through intelligent task distribution across multiple servers, they present considerable complexity in offloading strategy design, requiring sophisticated optimization frameworks to realize their full potential.
- Real-time decision making challenges: VEC systems require the real-time decision making due to the highly dynamic nature of vehicular networks. The computational complexity introduced by simultaneously handling multiple uncertainties makes it very challenging to develop algorithms that can provide near-optimal solutions within the tight delay constraints of VEC applications.

Furthermore, when both communication and computation uncertainties are considered together, the resulting optimization problem becomes significantly more complex. The mutual coupling between these two types of uncertainties creates a multi-dimensional uncertainty space that traditional optimization approaches cannot handle effectively. Specifically, the problem transforms from a standard mixed-integer nonlinear programming (MINLP) to a robust MINLP with coupled uncertainties, requiring sophisticated solution methodologies.

The motivation for this work stems from the critical gap between theoretical assumptions in existing literature and practical VEC deployment requirements. In real-world VEC scenarios, CAVs usually operate in highly dynamic environments where perfect CSI and accurate computation complexity estimation are practically impossible to achieve. The safety-critical nature of CAV applications demands highly reliable task offloading strategies that can handle multiple uncertainties concurrently while maintaining efficient resource utilization.

To fill this critical research gap and address the aforementioned challenges, our paper investigates a robust partial offloading scheme in a MEC-based vehicular network. By jointly optimizing task partitioning and task offloading decisions, we effectively reduce the total task processing delay for CAVs. Our approach provides a comprehensive solution that simultaneously addresses both communication and computation uncertainties while maintaining computational efficiency for real-time applications. Firstly, we conduct channel modeling with imperfect CSI. Then, to address computation uncertainty, we aim to find the optimal solution for the worst-case scenario. The main contributions of this paper are summarized as follows:

- Unlike existing work, we simultaneously consider both imperfect CSI and computation uncertainty in VEC, making our approach more practical for real-world scenarios. In the considered system, CAVs can partially offload their tasks to multiple MEC servers, which enhances the utilization of computational resources. Our objective is to minimize the system's total task processing delay. To this end, we model this problem as a MINLP.
- To solve this complex MINLP, we design an iterative framework to alternately optimize task partitioning and task offloading decisions. Specifically, we derive the task partitioning strategy by analyzing the characteristics of the optimal solution. Then, we transform the task offloading problem into a one-to-many matching problem. Through our proposed task offloading strategy based Gale-Shapley algorithm (TOGS), we obtain the optimal solution under a given task partitioning ratio.
- Simulation results show that our proposed approach reduces the system's total task processing delay by approximately 41% and 54% compared to the benchmark KM-Hybrid and KM-Edge schemes, respectively. Moreover, the proposed offloading scheme demonstrates its robustness across various computational complexity scenarios.

The rest of this paper is organized as follows: Section 2 provides a comprehensive review of related work in the area of task offloading for VEC scenarios. The system model and problem formulation is presented in Section 3. Then, the solution design is introduced in Section 4. Simulation results are presented in Section 5. Finally, conclusions are drawn in Section 6.

2. Related work

Generally speaking, existing task offloading methods in vehicular edge networks can be categorized into binary offloading and partial offloading. Binary offloading refers to executing the entire task either completely locally or fully offloading it to the edge server [11–14]. The authors in [11] investigated a three-layer computing network composed of a high altitude platform station, roadside units (RSUs), and vehicles, and designed a joint caching and computing offloading scheme to reduce the task processing latency for CAVs. Except offloading decisions, the work [12] also investigated the impact of transmission power and computational resource allocation on VEC system performance, and employed an alternating iteration method based on block descent to minimize an energy and delay-based cost function. In pursuit of computational resource expansion and ubiquitous service provisioning, MEC has been seamlessly incorporated into space-air-ground integrated networks (SAGIN). To tackle the intricate problems of computation offloading and resource allocation within SAGIN environments, authors in [13] introduced a novel traffic-aware lightweight hierarchical offloading methodology. Their approach presents an innovative architectural separation of SAGIN into communication access platforms and computation offloading platforms, while leveraging advanced deep reinforcement learning (DRL) techniques to accomplish optimal offloading strategies. Considering the personalized demands in smart communities, [14] proposed a personalized federated DRL based method to generate proper policies of binary computation offloading and resource allocation.

To alleviate the low resource utilization problem of binary offloading schemes, both [11,12] adopted multi-layer computing networks. However, resource scheduling in this multi-layer network architecture is exceptionally complex. Consequently, the resulting offloading scheme designs are highly intricate, posing challenges in meeting the stringent real-time requirements of VEC systems. While the works in [13,14] have demonstrated effective task offloading schemes by employing DRL methodologies, their singular objective for task offloading remains a limiting factor that impedes the comprehensive exploitation of edge computing resources.

On the other hand, partial offloading allows tasks to be decomposed into multiple subtasks, with the execution location of each subtask determined based on actual conditions [15–18]. This partial offloading approach offers greater flexibility, enabling dynamic adjustments according to factors such as network conditions, computational resource availability, and task characteristics, thereby significantly enhancing system performance and efficiency. By introducing non-orthogonal multiple access technology, [15,16] achieved a system

Table 1
Comparison of related works with our scheme.

Related works	Computation uncertainty	Channel uncertainty	Partial offloading	Multi-server	Optimization methods
Wu et al. [15]	✗	✗	✔	✔	Layered algorithm and distributed algorithm
Qian et al. [16]	✗	✗	✔	✔	Optimal pre-offloading decision and subtask migration
Ma et al. [17]	✗	✗	✔	✗	Lyapunov optimization and DRL-based algorithm
Liu et al. [18]	✗	✗	✔	✔	Heat-aware offloading and dynamic scheduling algorithm
Wu et al. [19]	✔	✗	✗	✗	Convex optimization and DRL-based algorithm
Wang et al. [20]	✔	✔	✗	✗	Alternate optimization
Qu et al. [21]	✔	✗	✗	✗	Min–Max robust optimization
Wang et al. [22]	✗	✔	✗	✗	Iterative optimization and successive convex approximation
Zhang et al. [23]	✗	✔	✔	✗	Coalition game and convex optimization
Fang et al. [24]	✗	✔	✔	✔	Closed-form solution and user association algorithm
Our scheme	✔	✔	✔	✔	Matching algorithm and alternate optimization

where a CAV can offload tasks to multiple MEC servers while using a single resource block, thereby improving the utilization of both computation and communication resources. A vehicular-cloud-assisted MEC network is proposed in [17] to fully explore the idle resources and ensure the stability of task queue. Each computation task is split into two portions and executed simultaneously at the local and server on vehicular cloud or MEC collaboratively. The authors in [18] improved the task offloading success rate and end-to-end latency by proposing a multi-MEC resource compensation method and a dynamic scheduling algorithm.

Nevertheless, these task offloading schemes in the aforementioned studies were all developed under the assumption that edge servers know the computational complexity of the offloaded tasks. In practical edge computing networks, edge servers cannot accurately estimate the computational complexity of pending offloaded tasks. This reality poses a significant challenge to task offloading decisions, as it increases system uncertainty and may lead to improper resource allocation or performance degradation.

To address computation uncertainty issue, many studies have adopted robust optimization (RO) approaches in designing task offloading schemes [19–21]. The authors in [19] proposed a robust distributed hierarchical online learning approach for resource allocation in dynamic MEC networks, enhancing algorithmic robustness and reducing recurring costs from network changes. By considering the worst-case scenario, the authors in [20] proposed a task-driven robust resource allocation algorithm, aiming to minimize the maximum task processing delay with system fairness as the objective. In [21], the authors studied the robust offloading scheduling problem for MEC by considering the uncertain offloading failures. Besides, recent advances in load prediction provide another complementary perspective to mitigate the impact of computation uncertainty. For instance, the federated deep learning framework proposed in [25] enables accurate and adaptive edge load prediction via personalized federated learning, which can support proactive resource provisioning under highly-variable environments.

While these approaches address computation uncertainty in general edge computing scenarios, VEC presents additional challenges. VEC not only inherits the computation uncertainty issues common to edge computing networks but also faces unique complications due to the high-speed movement of vehicles. Such as, due to high vehicle mobility and finite communication coverage of base stations, it is very challenging to maintain uninterrupted and high-quality services. To address this issue, a novel mobility-aware seamless service migration and resource allocation framework via convex-optimization-enabled DRL is proposed in [26]. Moreover, vehicle links are affected by the Doppler effect, complicating the acquisition of precise CSI and introducing channel uncertainty into VEC [22]. To tackle this issue, the authors in [23] jointly optimized offloading decisions and resource allocation to minimize task processing latency while considering uncertain channel conditions. And the authors in [24] proposed an energy-efficient resource allocation framework for a multi-user multi-BS MEC network with imperfect CSI.

Based on the previously discussed studies, it is evident that research on task offloading that simultaneously addresses both channel and computation uncertainties remains scarce, particularly in rapidly time-varying and latency-sensitive VEC scenarios. Furthermore, it is also found that many existing works adopt binary offloading schemes, which may lead to underutilization of computational resources on edge devices. The comparison of the related works and our scheme is further summarized in Table 1. Although all these works have made a great contribution to the task offloading design for VEC, they still face significant challenges in addressing the complex and dynamic nature of vehicular networks. The inherent uncertainties in channel state information, coupled with the computation uncertainty, demand more robust and adaptive task offloading strategies.

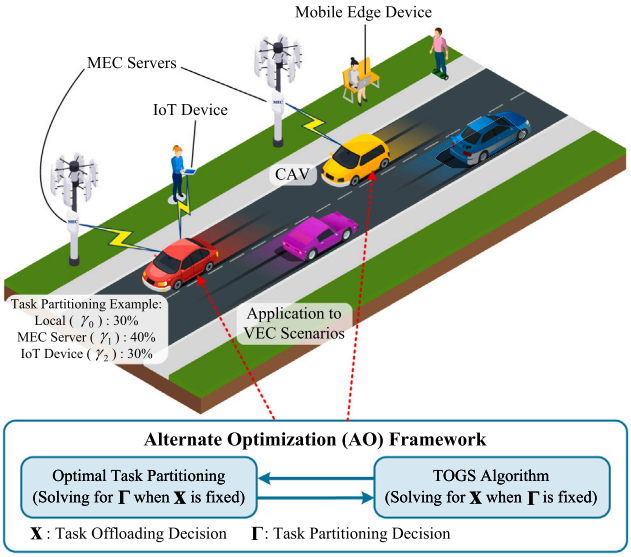


Fig. 1. Task offloading scenario in the VEC.

3. System model and problem formulation

As shown in Fig. 1, the target VEC scenario consists of M CAVs that need to process computing tasks onboard, and N MEC servers located outside the roads providing computing resources. The system employs orthogonal frequency division multiple access, with each subcarrier having a bandwidth of B. To efficiently utilize the computing resources of the MEC servers, CAVs can partition their tasks into multiple subtasks and offload them to multiple MEC servers through partial offloading techniques. As illustrated in Fig. 1, mobile devices may also act as potential computing service providers. To avoid high communication interference at the edge devices caused by task information from multiple CAVs, this paper considers that each MEC server can only provide computing resources to one CAV.

3.1. Communication model

The channel gain between CAV i and MEC server j is expressed as

$$H_{i,j} = L_{i,j} \left| g_{i,j} \right|^2, \tag{1}$$

where the large-scale fading model is given by $L_{i,j}(d)[dB] = 128.1 + 37.6log_{10}(d)$, with d is the distance from CAV i to MEC server j in kilometers. Considering the Doppler shift caused by the rapid relative movement between the CAV and MEC server, the small-scale fading information $g_{i,j}$ cannot be accurately obtained, and only its estimate $\tilde{g}_{i,j}$ is available [27]. Therefore, we model the small-scale fading channel estimation of $g_{i,j}$ by using the first-order Gauss–Markov process [28] as follows:

$$g_{i,j} = \lambda \tilde{g}_{i,j} + \sqrt{1-\lambda^2} e_{i,j}, \tag{2}$$

where $e_{i,j}$ is the distribution of estimation error. We assume $e_{i,j}$ is i.i.d and follows $\mathcal{CN}(0,1)$. Similarly, $\tilde{g}_{i,j}$ is also i.i.d as $\mathcal{CN}(0,1)$, and is independent of and uncorrelated with $e_{i,j}$. According to the Jakes statistical model for the fading channel [29], the coefficient λ $(0 < \lambda < 1)$ is given by

$$\lambda = J_0(2\pi f_D t_0), \tag{3}$$

where J_0 is the zero-order Bessel function of the first kind. $f_D = v f_c / c$ is the maximum Doppler shift, with v is the CAV speed, f_c is the carrier frequency, and c is the light speed. t_0 is the channel information feedback delay.

Therefore, when CAV i offloads tasks to MEC server j, its transmission rate $R_{i,j}$ is given by

$$R_{i,j} = B \log_2 \left(1 + \frac{P^v H_{i,j}}{N_0} \right), \tag{4}$$

where P^v is the CAV's transmit power, and N_0 is the channel noise power.

3.2. Computing model

This paper models the task of CAV i as $J_i = \{D_i, C_i\}$, where D_i is the task data size and C_i is the computational resources required for the task. The computational demand C_i is expressed as $C_i = D_i \omega_i$, where ω_i indicates the computational complexity of the task, measured in cycles per bit. Before executing the computing task, the task analyzer can obtain the exact value of the data size D_i, while the actual task complexity ω_i cannot be accurately acquired, i.e., there is computation uncertainty. This is consistent with the real-world situation where the size of a computing task is measurable, but its processing time is generally uncertain before processing [30]. Although the actual task complexity is uncertain, its expected value can be reasonably estimated based on long-term statistical data for the given task type. In this case, its statistical information can be used to characterize the uncertainty of task complexity [20]. In this paper, the actual task complexity ω_i and the estimated task complexity $\hat{\omega}_i$ can be modeled as

$$\omega_i = \hat{\omega}_i + \delta_i, |\delta_i| \le \epsilon_i, \tag{5}$$

where δ is the estimation error of task complexity, which is bounded within a given radius ϵ. We use the binary variable $x_{i,j}$ to represent whether CAV i offloads its task to MEC server j, where $x_{i,j} = 1$ indicates offloading and vice versa. The task processing delay T_i of CAV i consists of two parts: the local processing delay T_i^{loc} and the task offloading delay T_i^{edge}. The task partitioning decision of CAV i is denoted as $\Gamma_i = \{\gamma_{i,0}, \gamma_{i,1}, \dots \gamma_{i,N}\}$, where $\gamma_{i,0}$ is the local computing ratio. Clearly, we have

$$\gamma_{i,0} + \gamma_{i,1} + \cdots + \gamma_{i,N} = 1. \tag{6}$$

Then, the local processing delay of CAV i is

$$T_i^{loc} = \frac{\gamma_{i,0} C_i}{F_i}, \tag{7}$$

where F_i is the local computing power of CAV i.

The offloading delay $T_{i,j}^{edge}$ for CAV i to offload tasks to MEC server j consists of the task transmission delay $T_{i,j}^{tran}$ and the task computing delay $T_{i,j}^{com}$, which can be expressed as

$$T_{i,j}^{tran} = \frac{\gamma_{i,j} D_i}{R_{i,j}}, \tag{8}$$

$$T_{i,j}^{com} = \frac{\gamma_{i,j} C_i}{f_j}, \tag{9}$$

where f_j is the computing power of MEC server j. Then, the task offloading delay of CAV i is

$$T_i^{edge} = \max\{T_{i,1}^{edge}, \dots, T_{i,N}^{edge}\}. \tag{10}$$

3.3. Problem formulation

The task processing delay for all CAVs is the maximum of their local processing delay and task offloading delay. Then, the overall system task processing delay T_{tot} can be represented as

$$T_{tot} = \sum_{i=1}^{M} \max\{T_i^{edge}, T_i^{loc}\}. \tag{11}$$

Our optimization objective is to minimize the overall system task processing delay T_{tot} by optimizing the offloading decision $\mathbf{x} = \{x_{i,j} | i \in M, j \in N\}$ and task partitioning decision $\mathbf{\Gamma} = \{\Gamma_i | i \in M\}$. Thus, the problem is formulated as

$$\mathbf{P0} \quad \min_{\{\mathbf{x},\mathbf{\Gamma}\}} T_{tot}$$

$$\begin{aligned}
s.t. \quad & C1: \gamma_{i,j} \in [0,1], \forall i \in M, \forall j \in N; \\
& C2: \sum_{j=0}^{N} \gamma_{i,j} = 1, \forall i \in M; \\
& C3: x_{i,j} \in \{0,1\}, \forall i \in M, \forall j \in N; \\
& C4: \sum_{i \in M} x_{i,j} = 1, \forall j \in N; \\
& C5: R_{i,j} \ge R_{\min}, \forall i \in M.
\end{aligned}$$

where C1 and C2 are the constraints for the task partitioning ratios of CAVs, indicating that CAVs adopt a partial offloading approach. Constraints C3 and C4 ensure that each CAV can offload subtasks to multiple MEC servers, while each server can only provide computing services to a single CAV. C5 is the QoS constraint on the transmission rate of CAVs, indicating that task offloading cannot be performed when the transmission rate is below the lower bound $R_{\min}$.

4. Robust task offloading scheme design

Apparently, problem **P0** is a challenging uncertain optimization problem, where the task computational complexity C is an uncertain parameter. To make the optimization scheme designed in this paper more adaptable to different computational complexity scenarios, we adopt RO to solve **P0**. Generally speaking, the RO algorithm pursues the optimal solution under the worst-case scenario considering the impact of uncertainty. Therefore, we consider minimizing the overall system delay when the computational complexity is maximum, i.e., $\omega = \hat{\omega} + \epsilon$.

However, the discrete offloading decision variables and the continuous task partitioning decision variables are coupled, which leads to the problem **P0** being an extremely difficult MINLP problem. Traditional MINLP solvers (e.g., branch-and-bound) incur high computational complexity when solving problem **P0**, making them unsuitable for the low-latency requirements of vehicular networks. Furthermore, convex relaxation can be applied to binary variables but often produces fractional solutions that require rounding operations, which may

significantly degrade performance given the problem's structure. Metaheuristic methods like genetic algorithms or simulated annealing lack performance guarantees and require extensive tuning.

Therefore, to effectively solve the complex problem **P0**, we decouple it into two subproblems. First, we investigate how to achieve the optimal task partitioning ratio under given conditions for offloading decisions. Then, we propose TOGS algorithm to derive offloading decisions. Finally, through the designed iterative framework, we alternate between optimizing task partitioning and offloading decisions to approximate the optimal value.

4.1. Optimal task partitioning strategy

Assuming that $\mathbf{x}$ is fixed, then **P0** is equivalent to minimizing the delay of each CAV under the condition that MEC servers provide deterministic computing power. In this condition, for CAV i, the task processing latency reaches the minimum when the local computation latency of CAV i and the task offloading latency from each MEC server providing computation are equal. This property is formally established in the following proposition.

Proposition 1. *For the task partitioning optimization problem in CAV i, the optimal solution occurs when all processing delays are equal, i.e.,*

$$T_i^{loc} = T_{i,1}^{edge} = T_{i,2}^{edge} = \cdots = T_{i,N}^{edge} = T_i^* \tag{12}$$

where T_i^* is the optimal total delay for CAV i.

The detailed proof of Proposition 1 is provided in Appendix. Based on this theoretical foundation, we can derive the optimal task partitioning strategy as follows: When CAV i offloads tasks to n MEC servers, we can obtain the system of equations (for simplicity, we omit the subscript i here, such as $\gamma_{i,0}$ represented as γ_0), as follows:

$$\begin{cases} a_0\gamma_0 = a_1\gamma_1 = \cdots = a_n\gamma_n, \\ \gamma_0 + \gamma_1 + \cdots + \gamma_n = 1, \end{cases} \tag{13}$$

where a_n is defined as

$$a_n = \frac{D}{R_n} + \frac{C}{f_n}, \tag{14}$$

particularly, a_0 is the coefficient before local computation ratio, $a_0 = C/F$, hence every element in the vector $\mathbf{a} = (a_0, a_1, \ldots, a_n)^T$ is non-zero. Then, the Eq. (13) can be written in matrix form as follows:

$$\underbrace{\begin{pmatrix} a_0 & -a_1 & 0 & \cdots & 0 \\ a_0 & 0 & -a_2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_0 & 0 & \cdots & \cdots & -a_n \\ 1 & 1 & \cdots & 1 & 1 \end{pmatrix}}_{\mathbf{A}} \underbrace{\begin{pmatrix} \gamma_0 \\ \gamma_1 \\ \vdots \\ \gamma_{n-1} \\ \gamma_n \end{pmatrix}}_{\boldsymbol{\gamma}} = \underbrace{\begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \end{pmatrix}}_{\mathbf{b}}. \tag{15}$$

By performing matrix row transformations, we obtain $r(\mathbf{A}) = r(\mathbf{A}|\mathbf{b}) = n + 1$, thus, the system of equations $\mathbf{A}\boldsymbol{\gamma} = \mathbf{b}$ has a unique solution. Therefore, when the offloading decision is fixed, we can construct a system of equations based on the condition of equal task processing latencies at each computing end, and solve for a unique and optimal task partitioning ratio.

4.2. Task offloading strategy based TOGS

In this subsection, we discuss how to select the optimal offloading targets for all CAVs when the task partitioning ratios are known. This optimization problem can be modeled as follows:

$$\mathbf{P1} \quad \min_{\mathbf{x}} T_{tot}$$

$$s.t. \quad C3, C4, C5.$$

In this paper, each CAV can offload its tasks to multiple MEC servers, and within the transmission rate constraint, the task processing delay of each CAV is minimized as much as possible. Furthermore, to ensure the communication quality of the servers, the resources of each server can be shared with at most one CAV. We view CAVs and servers as two disjoint sets, and clearly **P1** can be regarded as a one-to-many two-sided matching problem. The one-to-many matching problem has been extensively studied, and Gale and Shapley, while solving the college admission problem, designed a stable one-to-many matching mechanism, namely the GS algorithm [31]. Through this mechanism, students and colleges are mutually matched, ensuring that there is no situation where a student A prefers a college B in another set, and the college B also prefers the student A over their respective matched college and student.

Inspired by the college admission problem, we propose an enhanced GS algorithm specifically tailored for the one-to-many task offloading matching problem proposed in this paper. The core of the algorithm lies in establishing suitable preference lists. The optimization objective in **P1** is to match the CAV matching set $\mathcal{V}$ and the MEC server matching set $\mathcal{C}$, such that the overall system delay is minimized while satisfying the transmission delay constraint. Therefore, we define a utility function matrix $\mathcal{U}$ to evaluate the task processing delay of all possible matchings between CAVs and MEC servers, thereby measuring the preferences of CAVs and MEC servers. Considering there are M CAVs and N MEC servers, the size of the matrix $\mathcal{U}$ is $M \times N$

$$\mathcal{U} = \begin{pmatrix} u_{1,1} & u_{1,2} & \cdots & u_{1,N} \\ u_{2,1} & u_{2,2} & \cdots & u_{2,N} \\ \vdots & \vdots & \ddots & \vdots \\ u_{M,1} & u_{M,2} & \cdots & u_{M,N} \end{pmatrix}. \tag{16}$$

Based on the optimal task partitioning results from the previous subsection, we calculate the reciprocal of the delay value as the utility function value $u_{i,j}$, which can be expressed as

$$u_{i,j} = \frac{1}{max\{T_i^{edge}, T_i^{loc}\}}. \tag{17}$$

The utility function matrix $\mathcal{U}$ provides preference lists for both sides of the matching set. For example, the preference list P_i of CAV i can be constructed as

$$P_i = \text{argsort}^{\downarrow}\{u_{i,j} \mid j \in N\}. \tag{18}$$

Based on the individual preferences of CAVs and MEC servers, the TOGS proposed in this paper can provide a stable and optimal match between CAVs and MEC servers. In TOGS, CAVs and MEC servers correspond to schools and students in the college admission problem, respectively. MEC servers send offloading invitations to CAVs, and CAVs have fixed offloading quotas.

The main process of the algorithm is shown in Algorithm 1. First, the algorithm receives the utility matrix under the current decisions and obtains the preference sequences of CAVs and servers based on the utility matrix. Then, servers send offloading invitations to the CAVs with the highest preference values. CAVs select the highest preferred server that meets the transmission rate constraints from all applicants to join their offloading queue. Unmatched servers will send offloading invitations to their next preferred CAV in the next round. The above steps are repeated until all servers are successfully matched. Finally, the algorithm outputs the optimal offloading decisions under the current utility matrix.

4.3. Complexity analysis of TOGS

In this part, we analyze the computational complexity of TOGS. For constructing preference lists for all M MEC servers, the computational complexity is $\mathcal{O}(NM\log M)$. Similarly, the complexity for constructing preference lists for all N CAVs is $\mathcal{O}(MN\log N)$. In the

Algorithm 1: TOGS for Offloading Target Selection.

Input: Utility function matrix $\mathcal{U}$;
Output: Selected offloading targets for all CAVs;

Denote E_{um} as the set of unmatched MEC servers;
Denote O_i as the offloading quotas of CAV i;
Denote P_j as the request vector of j, where if MEC server j sends an offloading invitation to CAV i, set $P_{i,j} = 0$; otherwise, set $P_{i,j} = 1$;
while $E_{um} \neq \varnothing$ *and* $P_j \neq 0$ **do**
  Select MEC server j from E_{um};
  Server j sends an offloading invitation to the highest priority CAV i in its preference list, set $P_{i,j} = 0$;
  if $O_i \geq 1$ **then**
    CAV i selects the highest priority server that meets the constraints from all applicants, and rejects the other applicants;
  else
    Server j sends an offloading invitation to its next preferred CAV;
  if *Server j successfully matches with CAV i* **then**
    $O_i = O_i - 1$;
    Remove server j from the list E_{um};
return The optimal CAVs offloading decisions under the current task division ratio.

best-case scenario, each MEC server might only need one request pairing operation, thus the optimal time complexity for the matching operation is $\mathcal{O}(N)$; in the worst-case scenario, each MEC server might need to propose to all M CAVs before finding a match, leading to $\mathcal{O}(MN)$ operations for all servers. Therefore, the total computational complexity of the TOGS algorithm is $\mathcal{O}(MN\log N + NM\log M + MN) = \mathcal{O}(MN\max\{\log M, \log N\})$. Apparently, the computational complexity of TOGS is affordable in practical edge intelligent network.

4.4. Alternate optimization framework

In this subsection, we combine the task partitioning and task offloading decisions proposed in the previous two subsections. By using an iterative alternating optimization approach, we aim to approximate the optimal solution. Since task partitioning and task offloading are mutually coupled, the utility matrix received by TOGS is determined by the results of the previous iteration. Therefore, how to reasonably update the utility matrix is key to ensuring that this alternating optimization algorithm (AO) converges quickly and approximates the optimal solution.

Before the first iteration of the AO, we assume that all CAVs must select one and only one MEC server as their offloading target. In this case, we calculate the utility function for all pairing situations and populate the utility matrix. Consequently, the initial values of the utility matrix are determined, and the algorithm enters the stable matching phase. Through Algorithm 1, the CAVs obtain their current optimal offloading targets. By optimal task partitioning, the task partition ratios and task processing delays of the CAVs are updated, thus updating the corresponding elements in the utility matrix for these matched pairs. For the pairs that are eliminated in the stable matching, we update them as follows:

For the set of MEC servers C_m matched with CAV i, we select server j from C_m, where the task computation ratio of server j satisfies $\gamma_{i,j} \geq \gamma_{i,k}$, $k \in C_m$. Then, for the set of MEC servers C_{um} not matched with CAV i, we assign a virtual task $\hat{J}_i$ to them, where the attributes of this virtual task are $\hat{J}_i = \gamma_{i,j} J_i$. From this, we calculate the delays required for all servers in C_{um} to process this virtual task and sequentially update their corresponding elements in the utility matrix. This process ensures that all elements in the utility matrix row corresponding to CAV i are updated.

By repeating the above operation for all CAVs, the utility matrix can be successfully updated. Through this process, the AO framework for task partitioning and task offloading is proposed, as shown in Algorithm 2.

Algorithm 2: AO for Task Partitioning and Offloading.

Input: Set of CAVs $\mathcal{V}$, set of MEC servers C, the error tolerance threshold T_{th};
Output: Total system task processing delay;

Initialize the offloading decision $\mathbf{x}$;
Initialize the utility matrix $\mathcal{U}$ according to Eq. (16);
Calculate the task processing delay T_{tot}^{tem} using the optimal task partitioning decision;
for $Iteration = 1, 2, \cdots$ **do**
  Call Algorithm 1 to obtain the current optimal task offloading decision;
  Recalculate the optimal task partitioning using the new offloading decision;
  Obtain the current total system task processing delay T_{tot}^{new};
  if $T_{tot}^{tem} - T_{tot}^{new} \leq T_{th}$ **then**
    break
  else
    $T_{tot}^{tem} = T_{tot}^{new}$;
  Update the utility matrix in Eq. (16);
return Total system task processing delay.

Our proposed iterative framework offers several advantages: (1) it leverages the problem structure by decoupling the optimization of task partitioning and offloading decisions, significantly reducing computational complexity; (2) it guarantees convergence to at least a local optimum due to the monotonic improvement in each iteration; (3) it provides a stable matching solution for the offloading decisions through the TOGS, which can ensure that no vehicle–server pair would prefer each other over their current assignments.

5. Simulation results

In this section, we evaluate the effectiveness of our proposed algorithm framework through simulation. We consider a road 500 m long and 22.5 m wide, with CAVs uniformly distributed on the road. All the MEC servers are uniformly distributed within a rectangular area 500 m long and 100 m wide, located 11.25 m away from one side of the adjacent road. The task data size D for all CAVs follows a uniform distribution $D \sim U[a - 100, a + 100]$, in units of Kbits. The estimated task complexity $\hat{\omega}$ is randomly distributed between 60 to 80 cycles/bit. The computing capacities of all MEC servers are uniformly distributed between 2 to 4 GHz, and the locally available computing resources F_i for all CAVs follow a Gaussian distribution, represented as $F_i \sim N(\mu, \sigma^2)$. Some communication parameters in our simulations were set according to [32,33]. The other simulation parameters are summarized in Table 2.

5.1. Comparison schemes

In this paper, we adopt a one-to-many task offloading approach and design a GS-based AO framework for it. To evaluate the performance of the proposed framework, we compares it with the one-to-one task offloading approach by introducing the following two comparative schemes:

Table 2
The simulation parameters settings.

Parameter	Meaning	Typical value
B	Subcarrier bandwidth	10 MHz [32]
P^v	CAVs transmit power	26 dBm [33]
N_0	Channel noise power	−110 dBm [32]
v	CAVs speed	[4∼24] m/s
M	Number of CAVs	[2∼10]
N	Number of MEC servers	[9∼21]
a	Mean value of task data	[200∼800] cycles/bit
ϵ	Task complexity fluctuation upper bound	[2∼20] cycles/bit
μ	Mean of CAV computing power	1 G cycles/s
σ	Standard deviation of CAV computing power	0.05 G cycles/s

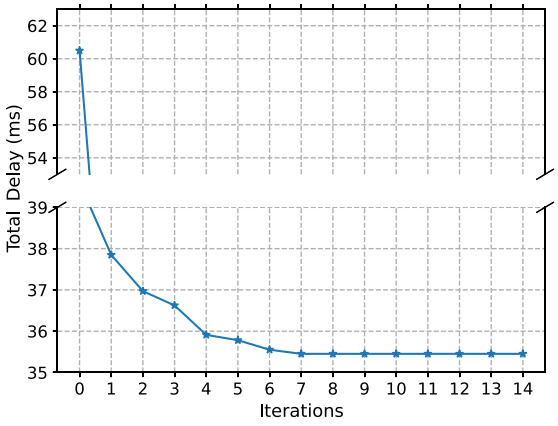


Fig. 2. Convergence of AO algorithm.

1. KM-Hybrid scheme: In this scheme, each vehicle can only select one MEC server as the offloading target. The task partitioning adopts the optimal task partitioning decision proposed in this paper, and the task offloading strategy uses the Kuhn–Munkres algorithm [34] that can find the optimal matching in the one-to-one matching problem. Therefore, KM-Hybrid can obtain the optimal solution in this one-to-one task offloading scenario.
2. KM-Edge scheme: Similar to the KM-Hybrid scheme, but this scheme adopts a full offloading approach, i.e., local CAVs do not participate in task computation and rely entirely on MEC servers.

It is worth noting that in the scenario considered in this paper, the KM-Hybrid scheme represents the optimal one-to-one offloading approach, while the KM-Edge scheme represents the optimal binary offloading approach.

5.2. Performance analysis

Fig. 2 demonstrates the convergence speed of the AO algorithm framework. In this simulation, the number of CAVs is 6, and the number of MEC servers is 15. In Fig. 2, when iterations is 0, it means the utility matrix is initialized, i.e., each CAV offloads tasks to only a single MEC server, so the total task processing delay of the system at this time is close to the KM-Hybrid scheme. After invoking Algorithm 1, the task offloading scheme switches to a one-to-many offloading, and the total task processing delay of the system decreases rapidly. It can also be seen that the AO algorithm framework only needs 6 rounds of iterations to converge, so the time complexity of this algorithm is relatively low, making it suitable for delay-sensitive V2X scenarios.

As shown in Fig. 3, we study the total task processing delay under different numbers of CAVs, while comparing the performance of the three schemes. In this simulation, the number of MEC servers is fixed at 15, and the mean task data size is fixed at 500 Kbits. It can be seen that as the number of CAVs increases, the total task load that the system needs to process also increases, resulting in a significant increase in delay for all three schemes. At the same time, the performance of our proposed scheme is better than KM-Hybrid. Specifically, the superior performance stems from our one-to-many offloading approach that enables better load balancing across multiple MEC servers. As the number of CAVs increases, the KM-Hybrid scheme suffers from suboptimal resource allocation because each CAV is constrained to use only one MEC server, leading to potential bottlenecks when some servers become overloaded while others remain underutilized. In contrast, our scheme can distribute the computational load more evenly by allowing each CAV to partition its tasks across multiple available servers. Furthermore, the KM-Hybrid scheme outperforms KM-Edge, and as the number of CAVs increases, the performance gap between KM-Edge and KM-Hybrid gradually widens, which is because in the KM-Edge scheme, local computing power is completely wasted. It is noteworthy that when the number of CAVs gradually approaches the number of MEC servers, our proposed scheme will gradually degenerate into a one-to-one offloading scheme, and its performance will approach KM-Hybrid scheme.

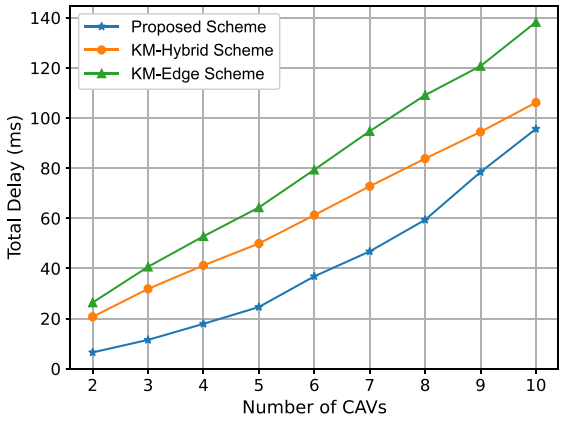


Fig. 3. Total delay vs. Numbers of CAVs.

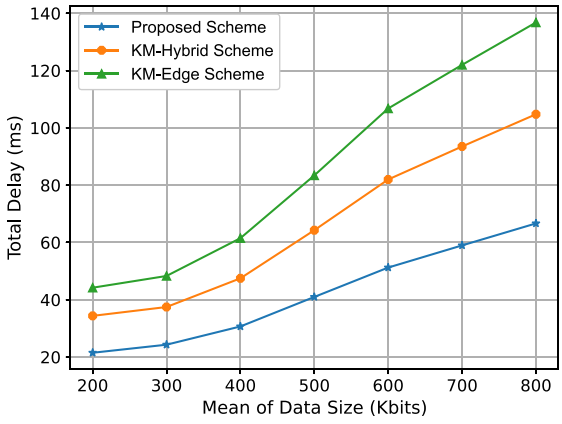


Fig. 4. Total delay vs. Mean of data size.

As illustrated in Fig. 4, we study the change in the total system delay under different task data sizes. In this simulation, the number of CAVs is 7, and the number of MEC servers is 15. From Fig. 4, we can see that the total processing delay increases as the task data size increases. This is because the task complexity is proportional to the task data size, so when the task data size becomes larger, the task computation delay increases. At the same time, an increase in the task data size also leads to an increase in the time required to transmit the task. Furthermore, the delay increase of our proposed scheme is obviously smaller than

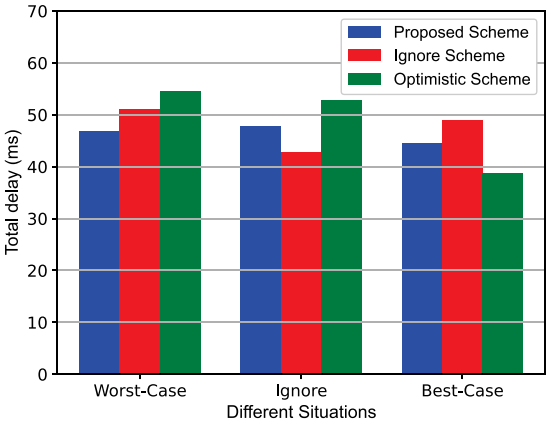


Fig. 5. Total delay vs. Different situations.

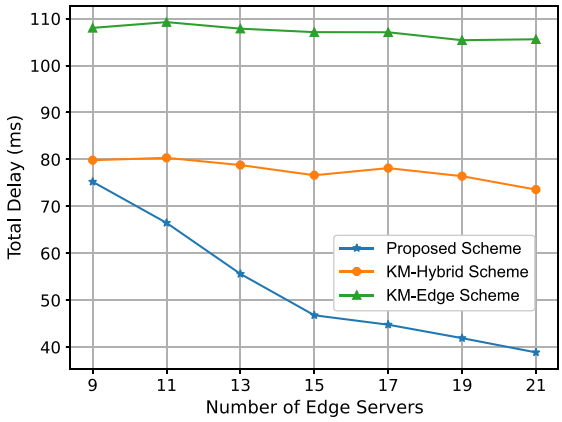


Fig. 7. Total delay vs. Number of edge servers.

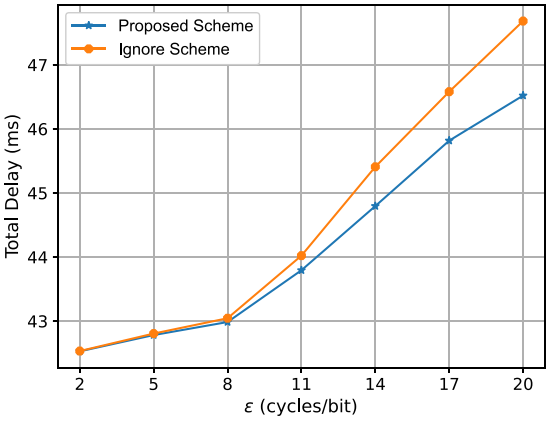


Fig. 6. The impact of task complexity fluctuation upper bound on total delay.

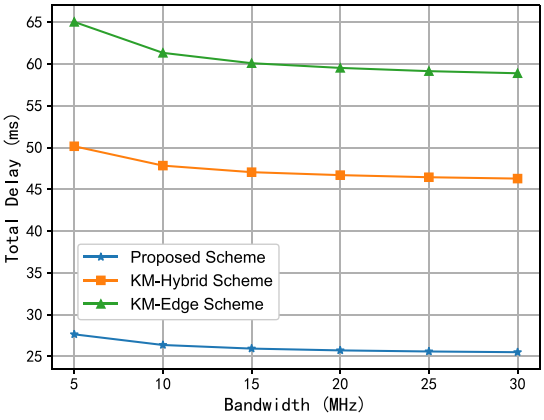


Fig. 8. Total delay vs. Bandwidth.

the other two comparative schemes, and the performance gap gradually widens as the task data size increases.

To study the robustness of the proposed scheme, we design two comparative schemes by modifying the proposed scheme: the optimistic scheme and the ignore scheme. The optimistic scheme considers the case where the computational complexity of the task is minimized, i.e., $\omega = \hat{\omega} - \epsilon$. The ignore scheme, which can also be considered a non-robust approach, treats the estimated task complexity as the actual task complexity. Meanwhile, these two schemes correspond to the "Best-Case" and "Ignore" scenarios, respectively, while our proposed scheme is based on the "Worst-Case" scenario. From Fig. 5, we can see that the three schemes perform optimally in their respective corresponding scenarios, especially the optimistic scheme in the "Best-Case" scenario, where its delay is minimized. However, its performance is the worst in the other two scenarios. In contrast, our proposed scheme exhibits stable performance across all three scenarios, making it more robust. It is noteworthy that both the optimistic scheme and the ignore scheme are modified based on the proposed algorithm. Therefore, if the system can obtain accurate computational complexity for the offloaded tasks, our proposed algorithm can quickly switch to the corresponding scheme for efficient resource utilization and further reduce the system delay.

Fig. 6 illustrates the impact of different upper bounds ϵ of task complexity fluctuation on the performance of the proposed scheme. As ϵ increases, the delay in both the proposed scheme and the ignore scheme increases. When ϵ is small, the actual task complexity is close to the estimated task complexity, and the performance of the two schemes is very similar. However, as ϵ further increases, the performance of the ignore scheme becomes significantly weaker than the proposed scheme that considers the worst-case scenario. This figure particularly highlights the robustness advantage of our approach. The ignore scheme's performance deteriorates rapidly as ϵ increases because it fails to account for computational uncertainty. In one-to-many partial offloading scenarios, the overall completion time is determined by the slowest processing component. Without considering worst-case complexity, the ignore scheme becomes vulnerable to unexpected computational delays that can significantly impact the entire system. In contrast, the proposed scheme, by considering the worst-case task complexity, can better cope with complexity fluctuations. Even when ϵ is large, the increase in task processing delay of the proposed scheme remains relatively slow. This robustness gives the proposed scheme an advantage in practical applications.

Fig. 7 shows the impact of the number of MEC servers on the total task processing delay of the system. In this simulation, the number of CAVs is fixed at 7, and the mean task data size is fixed at 500 Kbits. It can be seen that when the number of MEC servers is comparable to the number of CAVs, the performance of our proposed scheme is close to that of the KM-Hybrid scheme. However, as the number of MEC servers increases, our proposed scheme significantly outperforms the other two schemes. The KM-Hybrid and KM-Edge schemes can only select one MEC server, so their performance is not related to the number of edge servers, and thus their delay changes are not significant.

As illustrated in Fig. 8, we investigate the performance comparison of three schemes under different communication bandwidths. It can be observed that with increasing bandwidth, the total delay of all three schemes gradually decreases, and our proposed method demonstrates significant advantages under various communication bandwidth conditions. However, it is noteworthy that the declining trend of delay progressively slows down. This phenomenon occurs because bandwidth

increases only improve transmission rates, thereby reducing task transmission delay, but do not affect task computing delay. Therefore, when the bandwidth becomes sufficiently large, the total delay is primarily determined by computational delay rather than transmission delay.

6. Conclusions

This paper investigates how to design an efficient and robust MEC offloading scheme for CAVs in an urban vehicular network scenario, where the computing tasks are subject to uncertainty. To fully utilize the computational resources of edge servers, CAVs can partition their tasks into multiple subtasks and offload them to multiple MEC edge servers via orthogonal frequency division multiple access links, while each MEC server can only serve one CAV. For this, we model a one-to-many two-sided matching problem. And we designed a GS-based alternating optimization algorithm to solve this problem. Simulation results show that compared to the one-to-one offloading approach, our proposed scheme can better utilize the computing resources of edge servers, and our proposed scheme exhibits superior robustness. In future work, we will address the interdependencies among subtasks to further enhance the applicability of the offloading scheme.

CRediT authorship contribution statement

Pengcheng Qian: Investigation, Writing – original draft, Conceptualization, Formal analysis. **Liang Wang:** Project administration, Methodology, Supervision. **Jiamin Fan:** Writing – review & editing, Resources. **Zhenzheng Shi:** Writing – review & editing. **Mengge Li:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the National Natural Science Foundation of China under Grant 62071283.

Appendix. Proof of Proposition 1

1. Problem statement:

Given CAV i with task partitioning ratios $\boldsymbol{\gamma} = [\gamma_0, \gamma_1, \dots, \gamma_n]^T$, where $\gamma_j \geq 0$ and $\sum_{j=0}^{n} \gamma_j = 1$. We intend to minimize:

$$T_i = \max\{T_i^{loc}, T_{i,1}^{edge}, T_{i,2}^{edge}, \dots, T_{i,n}^{edge}\} = \max\{a_0\gamma_0, a_1\gamma_1, \dots, a_n\gamma_n\}, \tag{19}$$

where $a_j > 0$ are the delay coefficients for each computing option. Assume the optimal solution $\boldsymbol{\gamma}^*$ satisfies the condition that not all delay components are equal, i.e., there exist indices k and l such that:

$$a_k\gamma_k^* > a_l\gamma_l^*. \tag{20}$$

Without loss of generality, assume $a_k\gamma_k^* = \max_j\{a_j\gamma_j^*\} = T_i^*$ (the optimal objective value).

2. Construction of Better Solution:

Since $a_k\gamma_k^* > a_l\gamma_l^*$, we can transfer a small amount $\delta > 0$ from γ_k^* to γ_l^* while maintaining the constraint $\sum_{j=0}^{n} \gamma_j = 1$:

$$\tilde{\gamma}_k = \gamma_k^* - \delta, \tag{21}$$

$$\tilde{\gamma}_l = \gamma_l^* + \delta. \tag{22}$$

The new delays become:

$$a_k\tilde{\gamma}_k = a_k(\gamma_k^* - \delta) = a_k\gamma_k^* - a_k\delta, \tag{23}$$

$$a_l\tilde{\gamma}_l = a_l(\gamma_l^* + \delta) = a_l\gamma_l^* + a_l\delta. \tag{24}$$

To balance these delays, we can ensure that both components to have equal delay: $a_k\gamma_k^* - a_k\delta = a_l\gamma_l^* + a_l\delta$, now we have $\delta = \frac{a_k\gamma_k^* - a_l\gamma_l^*}{a_k + a_l} > 0$. With this choice of δ, both delays become: $a_k\tilde{\gamma}_k = a_l\tilde{\gamma}_l < a_k\gamma_k^*$. Therefore, we have $\max_j\{a_j\tilde{\gamma}_j\} < T_i^*$, contradicting the optimality of $\boldsymbol{\gamma}^*$.

3. Verification:

Constraint satisfaction: $\sum_{j=0}^{n} \tilde{\gamma}_j = 1 - \delta + \delta = 1$ holds.

Feasibility: For a sufficiently small δ, the inequality $\tilde{\gamma}_j \geq 0$ holds for all j.

4. Conclusion:

The contradiction proves that at the optimal solution, all delay components must be equal:

$$a_0\gamma_0^* = a_1\gamma_1^* = \cdots = a_n\gamma_n^* = T_i^*. \tag{25}$$

Data availability

Data will be made available on request.

References

[1] Steven E. Shladover, Connected and automated vehicle systems: Introduction and overview, J. Intell. Transp. Syst. 22 (3) (2018) 190–200.
[2] Wei Liu, Min Hua, Zhiyun Deng, Zonglin Meng, Yanjun Huang, Chuan Hu, Shunhui Song, Letian Gao, Changsheng Liu, Bin Shuai, Amir Khajepour, Lu Xiong, Xin Xia, A systematic survey of control techniques and applications in connected and automated vehicles, IEEE Internet Things J. 10 (24) (2023) 21892–21916.
[3] Siyu Teng, Xuemin Hu, Peng Deng, Bai Li, Yuchen Li, Yunfeng Ai, Dongsheng Yang, Lingxi Li, Zhe Xuanyuan, Fenghua Zhu, Long Chen, Motion planning for autonomous driving: The state of the art and future perspectives, IEEE Trans. Intell. Veh. 8 (6) (2023) 3692–3711.
[4] Hao Zhu, Ka-Veng Yuen, Lyudmila Mihaylova, Henry Leung, Overview of environment perception for intelligent vehicles, IEEE Trans. Intell. Transp. Syst. 18 (10) (2017) 2584–2601.
[5] Xiaolong Yang, Zesong Fei, Jianchao Zheng, Ning Zhang, Alagan Anpalagan, Joint multi-user computation offloading and data caching for hybrid mobile cloud/edge computing, IEEE Trans. Veh. Technol. 68 (11) (2019) 11018–11030.
[6] Sohan Gyawali, Yi Qian, Rose Qingyang Hu, Machine learning and reputation based misbehavior detection in vehicular communication networks, IEEE Trans. Veh. Technol. 69 (8) (2020) 8871–8885.
[7] Zheyi Chen, Jie Liang, Zhengxin Yu, Hongju Cheng, Geyong Min, Jie Li, Resilient collaborative caching for multi-edge systems with robust federated deep learning, IEEE Trans. Netw. 33 (2) (2025) 654–669.
[8] Nasir Abbas, Yan Zhang, Amir Taherkordi, Tor Skeie, Mobile edge computing: A survey, IEEE Internet Things J. 5 (1) (2018) 450–465.
[9] Junhui Zhao, Qiuping Li, Yi Gong, Ke Zhang, Computation offloading and resource allocation for cloud assisted mobile edge computing in vehicular networks, IEEE Trans. Veh. Technol. 68 (8) (2019) 7944–7956.
[10] Shaoshan Liu, Liangkai Liu, Jie Tang, Bo Yu, Yifan Wang, Weisong Shi, Edge computing for autonomous driving: Opportunities and challenges, Proc. IEEE 107 (8) (2019) 1697–1716.
[11] Qiqi Ren, Omid Abbasi, Gunes Karabulut Kurt, Halim Yanikomeroglu, Jian Chen, Caching and computation offloading in high altitude platform station (HAPS) assisted intelligent transportation systems, IEEE Trans. Wirel. Commun. 21 (11) (2022) 9010–9024.
[12] Farhan Pervez, Ajmery Sultana, Cungang Yang, Lian Zhao, Energy and latency efficient joint communication and computation optimization in a multi-UAV-assisted MEC network, IEEE Trans. Wirel. Commun. 23 (3) (2024) 1728–1741.
[13] Zheyi Chen, Junjie Zhang, Geyong Min, Zhaolong Ning, Jie Li, Traffic-aware lightweight hierarchical offloading toward adaptive slicing-enabled SAGIN, IEEE J. Sel. Areas Commun. 42 (12) (2024) 3536–3550.
[14] Zheyi Chen, Bing Xiong, Xing Chen, Geyong Min, Jie Li, Joint computation offloading and resource allocation in multi-edge smart communities with personalized federated deep reinforcement learning, IEEE Trans. Mob. Comput. 23 (12) (2024) 11604–11619.
[15] Yuan Wu, Kejie Ni, Cheng Zhang, Li Ping Qian, Danny H.K. Tsang, NOMA-assisted multi-access mobile edge computing: A joint optimization of computation offloading and time allocation, IEEE Trans. Veh. Technol. 67 (12) (2018) 12244–12258.
[16] Pengcheng Qian, Liang Wang, Yaguang Lin, Jiarong Du, Xiuxiu Dong, Joint power allocation and task offloading in NOMA enhanced MEC for ABS-assisted ITS, IEEE Commun. Lett. 27 (9) (2023) 2403–2407.

[17] Guifu Ma, Xiaowei Wang, Manjiang Hu, Wenjie Ouyang, Xiaolong Chen, Yang Li, DRL-based computation offloading with queue stability for vehicular-cloud-assisted mobile edge computing systems, IEEE Trans. Intell. Veh. 8 (4) (2023) 2797–2809.
[18] Jianhang Liu, Kunlei Xue, Qinghai Miao, Shibao Li, Xuerong Cui, Danxin Wang, Kewen Li, MCVCO: Multi-MEC cooperative vehicular computation offloading, IEEE Trans. Intell. Veh. 9 (1) (2024) 813–826.
[19] Yi-Chen Wu, Che Lin, Tony Q.S. Quek, A robust distributed hierarchical online learning approach for dynamic MEC networks, IEEE J. Sel. Areas Commun. 40 (2) (2022) 641–656.
[20] Qi Wang, Xiaoming Chen, Qiao Qi, Task-driven robust integration of communication and computation for edge-intelligent networks, IEEE Trans. Commun. 71 (1) (2023) 244–255.
[21] Yuben Qu, Haipeng Dai, Fan Wu, Dongyu Lu, Chao Dong, Shaojie Tang, Guihai Chen, Robust offloading scheduling for mobile edge computing, IEEE Trans. Mob. Comput. 21 (7) (2022) 2581–2595.
[22] Jun Wang, Daquan Feng, Shengli Zhang, An Liu, Xiang-Gen Xia, Joint computation offloading and resource allocation for MEC-enabled IoT systems with imperfect CSI, IEEE Internet Things J. 8 (5) (2021) 3462–3475.
[23] Lintao Zhang, Yanglong Sun, Yuliang Tang, Hao Zeng, Yuqi Ruan, Joint offloading decision and resource allocation in MEC-enabled vehicular networks, in: 2021 IEEE 93rd Vehicular Technology Conference, VTC2021-Spring, 2021, pp. 1–5.
[24] Fang Fang, Kaidi Wang, Zhiguo Ding, Victor C.M. Leung, Energy-efficient resource allocation for NOMA-MEC networks with imperfect CSI, IEEE Trans. Commun. 69 (5) (2021) 3436–3449.
[25] Zheyi Chen, Qingnan Jiang, Lixian Chen, Xing Chen, Jie Li, Geyong Min, MC-2PF: A multi-edge cooperative universal framework for load prediction with personalized federated deep learning, IEEE Trans. Mob. Comput. 24 (6) (2025) 5138–5154.
[26] Zheyi Chen, Sijin Huang, Geyong Min, Zhaolong Ning, Jie Li, Yan Zhang, Mobility-aware seamless service migration and resource allocation in multi-edge IoV systems, IEEE Trans. Mob. Comput. (2025) 1–17.
[27] Amen Memmi, Zouheir Rezki, Mohamed-Slim Alouini, Power control for D2D underlay cellular networks with channel uncertainty, IEEE Trans. Wirel. Commun. (2017) 1330–1343.
[28] Taejoon Kim, David J. Love, Bruno Clerckx, Does frequent low resolution feedback outperform infrequent high resolution feedback for multiple antenna beamforming systems? IEEE Trans. Signal Process. 59 (4) (2011) 1654–1669.
[29] Zhuwei Wang, Lihan Liu, Xiaodong Wang, Jing Zhang, Resource allocation in OFDMA networks with imperfect channel state information, IEEE Commun. Lett. (2014) 1611–1614.
[30] Shaowei Liu, Kaijun Ren, Kefeng Deng, Junqiang Song, A dynamic resource allocation and task scheduling strategy with uncertain task runtime on IaaS clouds, in: 2016 Sixth International Conference on Information Science and Technology, ICIST, 2016, pp. 174–180.
[31] D. Gale, L.S. Shapley, College admissions and the stability of marriage, Am. Math. Mon. 120 (2013) 386–391.
[32] Bo Li, Ruizhi Wu, Joint perception data caching and computation offloading in MEC-enabled vehicular networks, Comput. Commun. 199 (2023) 139–152.
[33] Xiaoshuai Li, Lin Ma, Yubin Xu, Rajan Shankaran, Resource allocation for D2D-based V2X communication with imperfect CSI, IEEE Internet Things J. 7 (4) (2020) 3545–3558.
[34] Xieli Fu, Chongtao Guo, Yanlin Qu, Xiao-Hui Lin, Resource allocation and blocklength selection for low-latency vehicular communications, IEEE Wirel. Commun. Lett. 10 (5) (2021) 914–918.



Pengcheng Qian received the B.E. degree in computer science and technology from Anhui University of Science and Technology, Huainan, China, in 2021. He is currently working toward the M.S. degree in computer science and technology with the School of Computer Science, Shaanxi Normal University, Xi'an, China. His research interests include robust resource allocation, Internet of Vehicles and mobile edge computing.



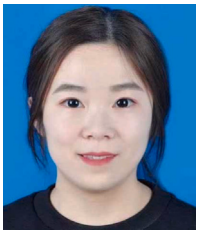
Liang Wang received the B.S. degree in telecommunications engineering and the Ph.D. degree in communication and information systems from Xidian University, China, in 2009 and 2015, respectively. He is currently an Associate Professor with the School of Computer Science, Shaanxi Normal University, Xi'an, China. From 2018 to 2019, he was a Visiting Scholar with the School of Electrical and Computer Engineering, Georgia Institute of Technology, USA. His research interests focus on Internet of Things, mobile edge computing, and applications of reinforcement learning and robust design in wireless communications networks.



Jiamin Fan received the B.E. degree in software engineering from Tianjin Normal University, Tianjin, China, in 2022. She is currently pursuing the M.S. degree in computer science and technology with the School of Computer Science, Shaanxi Normal University, Xi'an, China. Her current research interests include cell-free massive MIMO and deep reinforcement learning.



Zhenzheng Shi received the B.E. degree in computer science and technology from Qilu University of Technology (Shandong Academy of Sciences), Jinan, China, in 2023. He is currently working toward the M.E. degree in computer technology with the School of Computer Science, Shaanxi Normal University, Xi'an, China. His research interests include space-air-ground integrated networks, applications of reinforcement learning and MEC.



Mengge Li received the B.E. degree in information management and information systems from Shangqiu Normal University, Shangqiu, China, in 2019, and studied for the M.S. degree in computer applications technology from Shaanxi Normal University, Xi'an, China, from 2019 to 2022. She is currently studying for a Ph.D. degree in computer science and technology of Shaanxi Normal University, Xi'an, China. Her research interests include applications of reinforcement learning, and mobile crowdsensing.